DPS970 – Activity 1

Spam Filter Email Agent and Three-Jug Search Problem

Name: Minhaz Abedin
Student ID: 171424229
Date: October 10, 2026

All code, tests and test data are in this zip. `README.md` has how to run them. Implementation language: Python 3 (standard library only), structured after the textbook's pseudocode and the aima-python reference code.

Question 1 – Spam Filter Email Agent

1(a) PEAS description

	Spam filter agent
Performance measure	Every email ends up in the folder the rules require: allow-listed sender → `email/`; restrict-listed sender → `spam/`; more than 5 bad words in the body (sender not allow-listed) → `spam/`; otherwise → `email/`. Minimize false positives (legitimate mail sent to spam, the costliest error because the user may never see it) and false negatives (spam left in the email folder). Every email is processed exactly once (none lost, duplicated, overwritten or left behind). Speed / throughput.
Environment	The inbox folder of `.eml` files; the emails themselves (plain-text headers, bodies in different encodings: plain text, base64, quoted-printable, HTML, possibly encrypted; attachments); the allow list, restrict list and bad-word list; the `spam` and `email` destination directories on the file system.
Actuators	File-system operations that move (or copy) an email file into the `spam` directory or the `email` directory.
Sensors	A file reader for the `.eml` files in the inbox; a MIME parser/decoder that extracts the sender's address (and so its domain) from the `From` header and decodes the readable text of the body and of `text/*` attachments; a reader for the three list files.

1(b) Properties of the task environment

Property	Classification	Reason
Observable	**Partially observable**	The headers and text parts can be read, but encrypted or binary parts cannot, and the `From` header can be forged, so the sender's real identity and intent are hidden. However, every feature the rules use (sender domain, readable body text) is contained in the current percept, so the agent has what it needs for each decision.
Agents	**Single-agent**	Only the filter makes decisions in this environment. (In the wider world spammers adapt to filters, which would make it a competitive multi-agent setting, but they do not act while the agent is deciding.)
Deterministic / stochastic	**Deterministic**	The same email with the same lists always gets the same classification, and moving a file has a certain, known result.
Episodic / sequential	**Episodic**	Each email is an independent episode. Classifying one email does not change how the next one should be classified.
Static / dynamic	**Static**	The email being classified does not change while the agent decides. New mail arriving in the inbox does not affect the current decision.
Discrete / continuous	**Discrete**	A finite number of emails, percepts made of text, and only two actions, taken one email at a time.
Known / unknown	**Known**	The rules and the effect of each action are fully specified in advance.

1(c) Most appropriate agent type: simple reflex agent

A **simple reflex agent** (condition–action rules, AIMA Figure 2.10) is the best fit:

- **The decision depends only on the current percept.** Whether an email is spam is decided entirely from that one email: its sender domain and the number of bad words in its body. The activity's three rules are already condition–action rules
- **The environment is episodic.** Past emails never influence the current decision, so no internal state or memory is needed and a model-based (reflex-with-state) agent would add nothing. The tests confirm this: shuffling the order of the emails never changes any decision.
- **There is exactly one action per episode and it has no future consequences.** Nothing needs to be planned or searched, so a goal-based agent is unnecessary.
- **There is no trade-off to weigh.** The outcome is a crisp spam / not-spam decision fixed by the rules, so a utility-based agent is unnecessary.
- **The usual weakness of reflex agents does not apply.** Simple reflex agents fail when the needed information is not in the current percept. Although email in general is partially observable, every feature the rules test is in the percept.
- It is also fast, easy to test, and transparent: every decision can be explained by naming the rule that fired.

1(d) Implementation summary

Folder `q1_spam_filter/`. The structure follows AIMA Chapter 2 and aima-python's `agents.py`:

Textbook concept	**Code**
Agent = architecture + program	`Agent` class holding a `program` (`agents.py`)
Environment: percept / execute_action / step / run	`Environment` base class and `EmailEnvironment` (`environment.py`)
SIMPLE-REFLEX-AGENT	`SimpleReflexAgentProgram(rules, interpret_input)` (`agents.py`)
INTERPRET-INPUT	`make_interpret_input(...)` builds an `EmailState(sender_domain, on_allow_list, on_restrict_list, bad_word_count)` (`spam_agent.py`)
RULE-MATCH and condition–action rules	`rule_match` and `SPAM_FILTER_RULES` (`agents.py`, `spam_agent.py`)
Sensors	`read_eml` parses and decodes the `.eml` file (`email_percept.py`)
Actuators	`EmailEnvironment.execute_action` moves the file to `spam/` or `email/`

```
def program(percept):
    state  = interpret_input(percept)   # INTERPRET-INPUT
    rule   = rule_match(state, rules)   # RULE-MATCH
    action = rule.action                # rule.ACTION
    return action
```

Rules, checked in order (first match wins):

1. Sender domain on the **allow list** → `MoveToEmail`
2. Sender domain on the **restrict list** → `MoveToSpam`
3. **More than 5** bad-word occurrences in the body → `MoveToSpam`
4. Otherwise → `MoveToEmail`

Why it is rational: the performance measure is defined by these rules, and the agent applies exactly these rules to all the information it can perceive, so it always chooses the action that maximizes the performance measure.

Assumptions and design decisions

- The instructions describe the restrict list as "safe domains"; I assumed this as a typo for *unsafe* domains, since mail from them is spam.
- If a domain is on both lists, the allow list wins (it is checked first, and the bad-word rule explicitly exempts allow-listed mail).
- Sub-domains match: `noreply.github.com` matches the entry `github.com`. Look-alikes such as `evilgithub.com` do not.
- The domain comes from the real address, not the display name, so `"prof@senecapolytechnic.ca" <deals@evil.example>` is treated as `evil.example`.
- The "body" means all readable `text/*` parts, including text attachments. Base64 and quoted-printable are decoded, HTML tags and CSS are stripped, and a `multipart/alternative` email (the same text as plain + HTML) is read once so words are not double counted. Headers, including the Subject, are not part of the body. Non-text parts (PDFs, images, encrypted data) are unreadable and ignored.
- Bad words are matched case-insensitively, whole words only (`freedom` does not match `free`), and every occurrence counts. List entries with a space are phrases (`act now`). "More than 5" means 6 or more.
- Files are moved as the activity specifies; a `--copy` option keeps the inbox intact for demos. Existing files are never overwritten.

Testing: 43 unit tests and 19 test emails (`test_data/inbox`, expected folder for each in `test_data/expected_labels.csv`) cover each rule, the 5/6 threshold, sub-domains, spoofed display names, base64, quoted-printable, HTML, text and binary attachments, missing `From` header, bad words only in the subject, multipart/alternative, and whole-word matching. The agent places all 19 emails correctly (performance = 100%).

Question 2 – Three-Jug Search Problem

2(a) States, actions and transition model

State: a triple **(a, b, c)** = gallons currently in the 12-, 8- and 3-gallon jugs, with integers $0 \le a \le 12$, $0 \le b \le 8$, $0 \le c \le 3$. There are $13 \times 9 \times 4 = 468$ triples, of which **314 are reachable**: every action leaves at least one jug empty or full, so the $11 \times 7 \times 2 = 154$ triples with all three jugs partly filled can never occur.

Initial state: (0, 0, 0), all jugs empty.

Goal states: any state in which some jug holds exactly 1 gallon: $\{ (a, b, c) \mid a = 1 \text{ or } b = 1 \text{ or } c = 1 \}$. (If the goal also required the other jugs to be empty, two more Empty actions would be appended to the solution below.)

Actions (only those that change the state are applicable):

Action	**Applicable when**	**Meaning**
Fill(i)	jug i is not full	fill jug i from the faucet
Empty(i)	jug i is not empty	pour jug i onto the ground
Pour(i, j), $i \neq j$	jug i is not empty and jug j is not full	pour jug i into jug j until i is empty or j is full

With 3 jugs there are at most 3 + 3 + 6 = **12 actions** per state (branching factor $b \le 12$).

Transition model RESULT(s, a), with capacities C = (12, 8, 3):

- RESULT(s, Fill(i)): $s[i] \leftarrow C[i]$
- RESULT(s, Empty(i)): $s[i] \leftarrow 0$
- RESULT(s, Pour(i, j)): $t \leftarrow \min(s[i], C[j] - s[j])$; $s[i] \leftarrow s[i] - t$; $s[j] \leftarrow s[j] + t$

Action cost: 1 per action, so the path cost is the number of actions.

Solution found by BFS (3 actions, optimal):

Step	**Action**	**State**
0	(start)	(0, 0, 0)
1	Fill the 12-gal jug	(12, 0, 0)
2	Pour 12-gal into 8-gal	(4, 8, 0)
3	Pour 12-gal into 3-gal	(1, 8, 3) ✔

No state reachable in 2 or fewer actions contains 1 gallon (verified by a test), so 3 is the minimum.

2(b) Does a heuristic function make sense?

It can be defined, but it adds little for this problem.

The obvious idea, "how close is some jug to 1 gallon", is a poor guide. One pour can move up to 12 gallons, so being numerically close to 1 says little about how many actions remain.

A better informal heuristic comes from noticing that **Fill and Empty only ever produce an empty or a full jug (0, 12, 8 or 3), never 1 gallon**. The only way to create 1 gallon is a pour that leaves a remainder: pouring a jug holding x gallons into a jug with r gallons of room leaves $x - r = 1$ in the source. So:

- h(n) = 0 if some jug already holds 1 gallon;
- h(n) = 1 if a single pour would produce 1 gallon (some jug holds exactly one gallon more than the room left in another jug);
- h(n) = 2 otherwise.

This heuristic never overestimates (if no single action reaches the goal, at least two are needed), so it is admissible, and it is also consistent. It is implemented as `WaterJugsProblem.h`, and a test checks that it never exceeds the true cost for all 314 reachable states.

In practice, though, the heuristic is weak (it only takes the values 0, 1 and 2), and the space is tiny (314 states, $b \leq 12$, solution depth 3). Uninformed BFS finds the optimal answer instantly, so an informed search such as A* would not save meaningful work.

2(c) Is any search algorithm guaranteed to reach a goal state?

A goal is reachable (the 3-step solution above), so the question is whether a given algorithm will find it.

- **Every complete algorithm is guaranteed to reach a goal.** The state space is finite (314 reachable states), the branching factor is finite (≤ 12) and a solution exists. Therefore breadth-first search (complete when b is finite), uniform-cost search (all step costs equal 1), iterative deepening, and graph-search versions of DFS, greedy best-first search and A* (which keep a reached/closed set, so each of the 314 states is expanded at most once) must eventually generate a goal state. BFS also returns the *shortest* solution, because every action costs 1.
- **Not every algorithm, though.** The state graph has cycles (Fill 3 → Empty 3 returns to the start; pouring back and forth between two jugs repeats states). A tree-like depth-first search that does not check for repeated states can follow an infinite path such as Fill 3, Empty 3, Fill 3, Empty 3, … and never terminate. Depth-limited search with a limit below 3 returns *cutoff*, and greedy best-first tree search can also loop.
- If the goal were impossible (for example, measuring 1 gallon with 6- and 4-gallon jugs, where every amount is even), no algorithm could reach it. In that case BFS terminates and returns *failure* after exhausting the finite space; this case is in the test data.

2(d) Implementation summary

Folder `q2_water_jugs/`. It follows AIMA Chapter 3 and aima-python's `search4e` code:

Textbook concept	**Code (`search.py`, `jugs_problem.py`)**
Problem (initial, actions, result, is_goal, action_cost)	`Problem` base class, `WaterJugsProblem` subclass
Node (state, parent, action, path_cost)	`Node`
EXPAND	`expand(problem, node)`
BREADTH-FIRST-SEARCH	`breadth_first_search(problem)`
Solution extraction	`path_actions(node)`, `path_states(node)`

`breadth_first_search` is a line-by-line translation of Figure 3.9: a FIFO queue frontier, a `reached` set, and the goal test applied when a child is generated (early goal test), returning `failure` when the frontier empties.

Testing: 22 unit tests plus 11 test cases in `test_data/jug_cases.json`. These include the activity's problem, the classic 5/3 → 4 puzzle, other capacities and goals, a custom start state, a goal already true at the start, and two unsolvable cases. The tests check the formulation (actions, transitions, conservation of water, state-space size), that each returned solution replays correctly, that BFS solution lengths match an independent layer-by-layer check (so BFS is optimal), and that the heuristic is admissible.